

Your Tasks

Task 0: Preparation and Lab0 Design Document

1. Read the [Welcome to Pintos](#) and [GETTING STARTED ↗](#) chapter in Project Gitbook to setup your local development environment and get familiar with the course project.
2. Download the [project 0 design document template ↗](#). Read through it to motivate your design and fill it in after you finish the project.

Task 1: Booting Pintos

Exercise 1.1

1. Have Pintos development environment setup as described in [Enviroment Setup](#).
2. Afterwards, execute

```
cd pintos/src/threads
make
cd build
pintos --
```



If everything works, you should see Pintos booting in the [QEMU emulator ↗](#), and print `Boot complete.` near the end.

While by default we run Pintos in QEMU, Pintos can also run in the [Bochs ↗](#) and VMWare Player emulator. Bochs will be useful for the [Lab1: Threads](#). To run Pintos with Bochs, execute

```
cd pintos/src/threads
make
cd build
pintos --bochs --
```

✓ Exercise 1.1

Take screenshots of the successful booting of Pintos in QEMU and Bochs.

Exercise 2.1

While you are working on the projects, you will frequently use the GNU Debugger (GDB) to help you find bugs in your code. Make sure you read the [Debugging](#) section first.

In addition, if you are unfamiliar with **x86 assembly**, the [PCASM ↗](#) is an excellent book to start. Note that you don't need to read the entire book, just the basic ones are enough.

✓ Exercise 2.1

Your first task in this section is to **use GDB to trace the QEMU BIOS** a bit to understand how an IA-32 compatible computer boots. Answer the following questions in your design document:

- What is the first instruction that gets executed?
- At which physical address is this instruction located?

Exercise 2.2

✓ Exercise 2.2

Trace the Pintos bootloader and answer the following questions in your design document:

- How does the bootloader read disk sectors? In particular, what BIOS interrupt is used?
- How does the bootloader decide whether it successfully finds the Pintos kernel?
- What happens when the bootloader could not find the Pintos kernel?
- At what point and how exactly does the bootloader transfer control to the Pintos kernel?

Tips:

- In the second task, you will be tracing the Pintos bootloader. **Set a breakpoint at address `0x7c00`, which is where the boot sector will be loaded.** Continue execution until that breakpoint.
- Trace through the code in `threads/loader.S`, using the source code and the disassembly file `threads/build/loader.asm` to keep track of where you are.
- Also, **use the `x/i` command** in GDB to disassemble sequences of instructions in the boot loader, and compare the original boot loader source code with both the disassembly in `threads/build/loader.asm` and GDB.



Exercise 2.3

Suppose we are interested in **tracing the behavior of one kernel function** `palloc_get_page()` **and one global variable** `uint32_t *init_page_dir`. For this exercise, you do not need to understand their meaning and the terminology used in them. You will get to know them better in Lab3: Virtual Memory.

✓ Exercise 2.3

Trace the Pintos kernel and answer the following questions in your design document:

- At the entry of `pintos_init()`, what is the value of the expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?
- When `palloc_get_page()` is called for the first time,
 - what does the call stack look like?
 - what is the return value in hexadecimal format?
 - what is the value of expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?
- When `palloc_get_page()` is called for the third time,
 - what does the call stack look like?
 - what is the return value in hexadecimal format?
 - what is the value of expression `init_page_dir[pd_no(ptov(0))]` in hexadecimal format?

 The GDB command `p` may be helpful.

Tips:

- After the Pintos kernel takes control, the initial setup is done in assembly code `threads/start.S`.
- Later on, the kernel will finally kick into the C world by calling the `pintos_init()` function in `threads/init.c`.



- **Set a breakpoint at `pintos_init()`** and then continue tracing a bit into the C initialization code. Then read the source code of `pintos_init()` function.

Task 3: Kernel Monitor

Exercise 3.1

At last, you will get to make a small enhancement to Pintos and write some code!

- In particular, when Pintos finishes booting, it will check for the supplied command line arguments stored in the kernel image. Typically you will pass some tests for the kernel to run, e.g., `pintos -- run alarm-zero`.
- If there is no command line argument passed (i.e., `pintos --`, note that `--` is needed as a separator for the pintos perl script and is not passed as part of command line arguments to the kernel), the kernel will simply finish up. This is a little boring.

Your task is to add a tiny kernel shell to Pintos so that when no command line argument is passed, it will run this shell interactively.

- Note that this is a kernel-level shell. In later projects, you will be enhancing the user program and file system parts of Pintos, at which point you will get to run the regular shell.
- **You only need to make this monitor very simple.** Its requirements are described below.



✓ Exercise 3.1

Enhance **threads/init.c** to implement a tiny kernel monitor in Pintos.

Requirments:

- It starts with a prompt `PKUOS>` and waits for user input.
- **As the user types in a printable character, display the character.**
- When a newline is entered, it parses the input and checks if it is `whoami`. If it is `whoami`, print your student id. Afterward, the monitor will print the command prompt `PKUOS>` again in the next line and repeat.
- If the user input is `exit`, the monitor will quit to allow the kernel to finish. For the other input, print `invalid command`. Handling special input such as backspace is not required.
- If you implement such an enhancement, mention this in your design document.

i The code place for you to add this feature is in line 136 of `threads/init.c` with

```
// TODO: no command line passed to kernel. Run interactively.
```

i You may need to use some functions provided in **lib/kernel/console.c**, **lib/stdio.c** and **devices/input.c**.

[Previous
Background](#)

[Next
FAQ](#)

Last updated 1 year ago

